NUMU API - Days 31-60: Make It Reliable

2-Week Sprint | Yousef & Yahia

Team:

Yousef - Security, image pipeline & onboarding logic
Yahia - Observability, notifications & load testing

WEEK 1: Security + Observability

Current State: No PostgreSQL RLS | No 2FA/MFA | No security headers | No Sentry | Basic string logging (no JSON) | No staging docker-compose | Audit service interface defined but incomplete

Yousef - Security & Hardening

#	Task	Files to Create / Modify
1	Create PostgreSQL RLS policies for tenant-scoped tables (stores, products, orders, customers, categories, invoices, addresses, coupons)	alembic/versions/xxx_add_rls_policies.py
2	Create RLS helper function: set_tenant_context() called before each query + update DB connection to SET app.current_tenant	src/infrastructure/database/connection.py (modify), src/infrastructure/tenancy/rls.py
3	Add RLS integration tests - verify cross-tenant data isolation at DB level	tests/security/test_rls_isolation.py
4	Create 2FA entity + TOTP service using pyotp (generate secret, generate QR URI, verify code)	src/core/entities/two_factor.py, src/infrastructure/external_services/totp_service.py
5	Create 2FA use cases: Enable2FA (return QR + backup codes), Verify2FA, Disable2FA + generate 10 backup codes on enable	src/application/use_cases/auth/two_factor/
6	Create 2FA API routes: POST /auth/2fa/enable, POST /auth/2fa/verify, DELETE /auth/2fa/disable + add 2FA check in login flow	src/api/v1/routes/auth.py (modify), src/api/v1/schemas/auth.py (modify)
7	Create security headers middleware: X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, CSP, X-XSS-Protection, Referrer-Policy, Permissions-Policy	src/api/middleware/security_headers.py
8	Register security headers middleware in app + add security tests (verify headers present, 2FA flow, RLS bypass attempts)	src/main.py (modify), tests/security/test_security_headers.py, tests/security/test_2fa.py

Yahia - Observability & Staging

#	Task	Files to Create / Modify
1	Install sentry-sdk + initialize Sentry in app startup with environment, release, traces_sample_rate + capture unhandled exceptions	pyproject.toml (modify), src/main.py (modify), src/config/settings.py (modify)
2	Add Sentry middleware: capture request context (user_id, tenant_id, path) as tags + performance transaction tracking	src/api/middleware/sentry_middleware.py
3	Replace string logging with structlog: JSON formatter, bound loggers with request_id, tenant_id, user_id context propagation	src/api/middleware/logging.py (rewrite), src/config/logging_config.py
4	Add structured log calls across key flows: auth (login/register/2fa), orders (create/status), payments (webhook), errors	src/application/use_cases/ (modify multiple), src/api/v1/routes/webhooks/ (modify)
5	Enhance health check: add DB connectivity, Redis ping, Sentry DSN status, disk space + return detailed JSON status	src/api/v1/routes/health.py (modify)
6	Create docker-compose.staging.yml: Nginx reverse proxy + SSL termination, PostgreSQL with WAL archiving, Redis sentinel, API with staging env vars	docker/docker-compose.staging.yml, docker/nginx/nginx.conf, docker/nginx/ssl/

#	Task	Files to Create / Modify
7	**Create .env.staging template + staging deployment script + update Makefile with staging commands**	**.env.staging, scripts/deploy_staging.sh, Makefile (modify)**
8	Write observability tests: verify Sentry captures errors, structured logs contain required fields, health endpoint returns all checks	tests/integration/test_observability.py

WEEK 2: Media + Notifications + Onboarding

Current State: Image upload exists (R2) but NO optimization/resize | WhatsApp service IMPLEMENTED (template-based) but NOT wired to order events | Resend email service IMPLEMENTED | No onboarding wizard | No load tests

Yousef - Image Pipeline & Onboarding

#	Task	Files to Create / Modify
1	Create ImageProcessor service using Pillow: resize (thumbnail 150px, medium 600px, large 1200px), compress quality 85%, convert to WebP, strip EXIF metadata	src/infrastructure/external_services/image/image_processor.py
2	Create image optimization pipeline: on upload -> optimize original + generate 3 size variants + upload all to R2 + return URLs dict	src/infrastructure/external_services/image/image_pipeline.py
3	Integrate image pipeline into UploadProductImageUseCase: replace direct upload with pipeline, store variant URLs in product.media_urls	src/application/use_cases/products/upload_image.py (modify)
4	Create Celery task for async image processing (background optimization for bulk uploads)	src/infrastructure/messaging/tasks/image_tasks.py
5	Create OnboardingStep entity + MerchantOnboardingUseCase: track steps (create_store, add_product, configure_payment, add_shipping, first_order) with completion %	src/core/entities/onboarding.py, src/application/use_cases/stores/onboarding.py
6	Create onboarding progress routes: GET /stores/{store_id}/onboarding (get progress), POST /stores/{store_id}/onboarding/skip/{step}	src/api/v1/routes/stores/onboarding.py, src/api/v1/schemas/tenant/onboarding.py
7	Auto-trigger onboarding step completion: hook into create_store, create_product, configure_payment use cases to mark steps done	src/application/use_cases/stores/ (modify), src/application/use_cases/products/ (modify)
8	Write tests for image pipeline (resize dimensions, WebP output, EXIF stripped) + onboarding flow tests	tests/unit/test_image_pipeline.py, tests/unit/test_onboarding.py

Yahia - Notifications & Load Testing

#	Task	Files to Create / Modify
1	Wire WhatsApp to order events: send order_confirmation on checkout, shipping_update on ship(), delivery_confirmation on deliver()	src/application/use_cases/orders/checkout.py (modify), src/application/use_cases/orders/update_order_status.py (modify)
2	Wire email notifications to order events: order confirmation email, shipping notification email, delivery email via Resend service	src/application/use_cases/orders/checkout.py (modify), src/application/use_cases/orders/update_order_status.py (modify)
3	Create Celery tasks for async notification dispatch (don't block order flow): send_order_email_task, send_whatsapp_task	src/infrastructure/messaging/tasks/notification_tasks.py
4	Create notification preferences: allow customers to opt-in/out of WhatsApp + email per event type + store in customer profile	src/core/entities/customer.py (modify), src/api/v1/routes/storefront/customer.py (modify), alembic/versions/xxx_add_notification_prefs.py
5	Create welcome email + onboarding email sequence: send on merchant registration, 1st product added, 1st order received (Celery scheduled)	src/infrastructure/messaging/tasks/onboarding_email_tasks.py, src/infrastructure/external_services/resend/email_templates/ (new templates)
6	Create Locust load test suite: auth flow, browse products, add to cart, checkout, order status update - target 100 concurrent merchants	tests/load/locustfile.py, tests/load/README.md

#	Task	Files to Create / Modify
7	**Create Locust config with different profiles: smoke (10 users), load (100 users), stress (500 users) + Makefile targets**	**tests/load/locust.conf, Makefile (modify)**
8	Write notification tests: verify WhatsApp called on order events, email sent, notification preferences respected, load test smoke run	tests/integration/test_notifications.py, tests/integration/test_notification_prefs.py

PR Dependency Order (No Conflicts)

File scope column shows which directory each PR touches - PRs touching different directories cannot conflict.

Week 1 PRs: Security + Observability

PR	PR Name	Depends On	Owner	Files Touched
#20	feat: PostgreSQL RLS policies + tenant context helper	None (independent)	Yousef	alembic/, tenancy/
#21	feat: security headers middleware	None (independent)	Yousef	middleware/
#22	feat: Sentry SDK integration + middleware	None (independent)	Yahia	main.py, config/
#23	feat: structured logging with structlog (JSON)	None (independent)	Yahia	middleware/logging
#24	feat: 2FA entity + TOTP service + backup codes	None (independent)	Yousef	core/entities/, ext_svc/
#25	feat: 2FA use cases + API routes (enable/verify/disable)	PR #24	Yousef	use_cases/auth/, routes/
#26	feat: enhanced health check (DB, Redis, Sentry status)	PR #22	Yahia	routes/health.py
#27	feat: structured logs in key flows (auth, orders, payments)	PR #23	Yahia	use_cases/, webhooks/
#28	infra: docker-compose.staging + Nginx + .env.staging + deploy script	None (independent)	Yahia	docker/, scripts/
#29	test: RLS isolation + security headers + 2FA + observability	PR #20, #21, #25, #27	Yousef	tests/

Week 2 PRs: Media + Notifications + Onboarding

PR	PR Name	Depends On	Owner	Files Touched
#30	feat: ImageProcessor service (resize, WebP, compress, strip EXIF)	None (independent)	Yousef	ext_svc/image/
#31	feat: image optimization pipeline + Celery async task	PR #30	Yousef	ext_svc/image/, tasks/
#32	feat: integrate pipeline into product image upload	PR #31	Yousef	use_cases/products/
#33	feat: wire WhatsApp + email to order events via Celery tasks	None (independent)	Yahia	use_cases/orders/, tasks/
#34	feat: notification preferences (opt-in/out) + migration	None (independent)	Yahia	entities/customer, alembic/
#35	feat: onboarding entity + wizard use case + progress tracking	None (independent)	Yousef	core/, use_cases/stores/
#36	feat: onboarding routes + auto-trigger step completion	PR #35	Yousef	routes/stores/, schemas/
#37	feat: welcome + onboarding email sequence (Celery scheduled)	PR #34	Yahia	tasks/, resend/templates/
#38	feat: Locust load test suite (smoke/load/stress profiles)	None (independent)	Yahia	tests/load/
#39	test: image pipeline + onboarding + notifications + load smoke	PR #32, #36, #33, #38	Both	tests/

Parallel Work Timeline

Week 1:

Yousef:

```
[PR#20 RLS] --> [PR#21 Sec Headers] --> [PR#24 2FA entity] --> [PR#25 2FA routes] --> [PR#29 Tests]
 Day 1-2         Day 2                  Day 3                  Day 4                  Day 5
```

Yahia:

```
[PR#22 Sentry] --> [PR#23 Structlog] --> [PR#26 Health] --> [PR#27 Log flows] --> [PR#28 Staging]
 Day 1              Day 2                 Day 3              Day 4                Day 5
```

Week 2:

Yousef:

```
[PR#30 ImgProcessor] --> [PR#31 Pipeline] --> [PR#32 Upload] --> [PR#35 Onboarding] --> [PR#36 Routes]
 Day 1                    Day 2               Day 3              Day 3-4                Day 4-5
```

Yahia:

```
[PR#33 Wire notifs] --> [PR#34 Prefs] --> [PR#37 Onboard emails] --> [PR#38 Locust] --> [PR#39 Tests]
 Day 1-2                Day 2              Day 3                     Day 4              Day 5
```

Conflict-Free File Ownership Map

Yousef Owns (exclusive)	Yahia Owns (exclusive)
alembic/ (RLS migration only)	src/config/settings.py, logging_config.py
src/infrastructure/tenancy/rls.py	src/api/middleware/logging.py (rewrite)
src/api/middleware/security_headers.py	src/api/middleware/sentry_middleware.py
src/core/entities/two_factor.py	src/api/v1/routes/health.py
src/core/entities/onboarding.py	docker/docker-compose.staging.yml, nginx/
src/infrastructure/external_services/totp_service.py	src/infrastructure/messaging/tasks/notification_tasks.py
src/infrastructure/external_services/image/	src/infrastructure/messaging/tasks/onboarding_email_tasks.py
src/application/use_cases/auth/two_factor/	tests/load/
src/application/use_cases/stores/onboarding.py	.env.staging, scripts/deploy_staging.sh
src/api/v1/routes/stores/onboarding.py	src/core/entities/customer.py (notif prefs)

Shared Files (merge sequentially - never in parallel):

- src/main.py - Yousef adds security_headers middleware (PR#21), then Yahia adds Sentry init (PR#22). PR#22 rebases on PR#21.
- src/application/use_cases/orders/ - Yahia modifies checkout.py & update_status.py for notifications (PR#33). Yousef does NOT touch these.
- Makefile - Yahia adds staging commands (PR#28), then load test targets (PR#38). Both are Yahia's so no conflict.
- pyproject.toml - Yahia adds sentry-sdk + structlog (PR#22,#23). Yousef adds pyotp (PR#24). Merged sequentially, different lines.
- tests/ - PR#29 (Yousef) and PR#39 (Both) touch different test files. No overlap.